CS3081 – Cloud Computing
Mini Project Report

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Name: **Mukesh Kumar Verma**

Roll number: **230101006**

Programme: B.Tech, Computer Science and Engineering

Institute: IIIT Manipur

Course: CS3081 – Cloud Computing

GitHub repository:
https://github.com/mukeshpatel1006/cloud-command-parser-Project

Abstract

This project builds a small command-parsing system on a local cloud. A larger teacher model, Phi-3.5-mini-instruct run through Ollama, labelled 3,000 natural-language commands using two parallel workers. After cleaning, 2,765 valid examples remained. These examples were used to train a 70-million-parameter student, EleutherAI Pythia-70M, by sequence-level knowledge distillation. The student reads a command and returns a JSON object with the keys intent, date, time and place. On the 100-command validation set the student gave valid JSON every time and reached 48% full-frame accuracy, while the teacher reached 49%. The model is kept in SeaweedFS object storage, served by a FastAPI service in a Docker container, and deployed on a k3d Kubernetes cluster with two replicas, health probes, a Service and an Ingress. The report also covers subnet planning, a network ACL checked on six packets, eight passing unit tests, linting, secret scanning and a GitHub Actions workflow. Load, cost and autoscaling measurements were not completed, and they are marked as such instead of being estimated.

1. Introduction and System Architecture

1.1 Problem statement

A voice assistant has to turn a spoken sentence into something a program can use. This project takes one short English command, such as “set an alarm for seven thirty am tomorrow”, and returns a JSON object with four keys: intent, date, time and place. The model doing this must be small enough to run on an ordinary laptop.

1.2 Objectives

- Label 3,000 commands with a larger teacher model and clean the result.
- Train a 70-million-parameter student from the teacher’s answers.
- Keep data and model files in object storage, not in Git or inside the container image.
- Serve the student through a containerised API and run it on Kubernetes.
- Check the network rules, tests, CI/CD pipeline, performance and cost.

1.3 Technologies used

Area	Tools
Models	Phi-3.5-mini-instruct (Ollama), EleutherAI Pythia-70M, PyTorch, Hugging Face Transformers
Storage and cluster	SeaweedFS (S3 interface), k3d, Kubernetes, Traefik Ingress
Service	Python, FastAPI, Docker
Quality and delivery	pytest, Ruff, gitleaks, GitHub Actions, Locust

1.4 Complete system flow

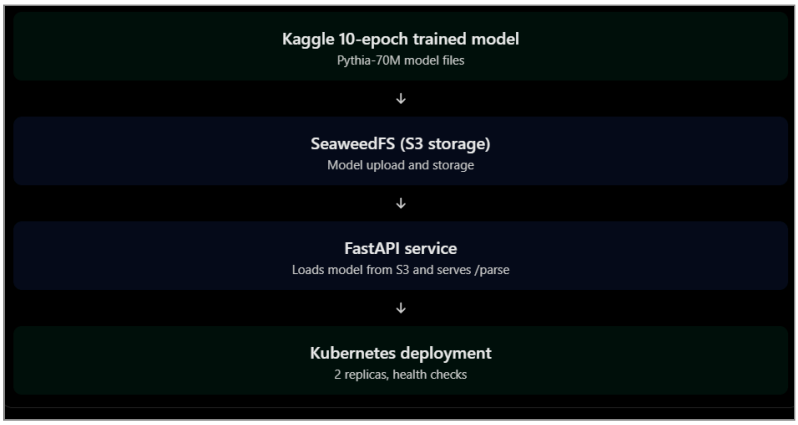


Figure 1. System flow: trained model, object storage, FastAPI service and Kubernetes deployment.

2. Task 1: Object Storage and Parallel Teacher Labelling

SeaweedFS runs inside the k3d cluster, in the namespace called data, and offers an S3 interface. I created two buckets there: datasets and models. The file commands_pool.jsonl, which holds the 3,000 commands, was uploaded to datasets.

Labelling was done by two workers running at the same time. Each worker reads two settings, SHARD_INDEX and SHARD_COUNT, and keeps the lines whose number leaves the remainder SHARD_INDEX when divided by SHARD_COUNT. This gave two shards of 1,500 commands, and every command went to exactly one worker. Each worker sent its commands to the teacher, Phi-3.5-mini-instruct through Ollama, with temperature 0 and seed 42, and saved the replies unchanged in its own file.

A cleaning script then removed replies that were not valid JSON, had the wrong keys, used an unknown intent, or contained a slot value that does not appear in the command. Duplicate commands were also removed. The final dataset has 2,765 examples.

Input	Not JSON	Wrong keys	Bad intent	Bad slot	Duplicates	Kept
3,000	2	14	7	165	47	2,765

The numbers add up: 2 + 14 + 7 + 165 + 47 + 2,765 = 3,000. About 92% of the records were kept. Most rejections were slot values that the teacher did not copy exactly from the command. I did not save a separate check that every command id appears in only one shard.

3. Task 2: Student Model Training and Distillation

The teacher is Phi-3.5-mini-instruct, with about 3.8 billion parameters. The student is EleutherAI Pythia-70M, with 70 million. I used sequence-level distillation: each training example is the text “Command: …” followed by “JSON: …”, where the JSON is the teacher’s own reply, and the student learns to write that reply. The teacher’s probabilities were not used, because Ollama only returns text and the two models do not share a vocabulary.

3.1 Training configuration

Hardware	Kaggle T4 GPU	Training examples	2,765
Epochs	10	Batch size	2
Optimiser	AdamW, FP32	Gradient clipping	0.5
Training time	about 7.23 minutes	Saved as	model-v2.tar.gz

3.2 Training flow

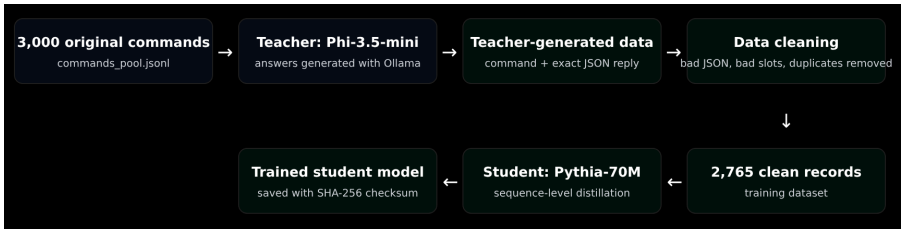


Figure 2. Task 2 flow from the original commands to the trained student.

3.3 Loss curve

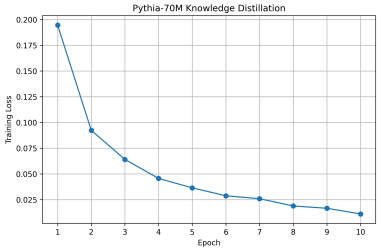


Figure 3. Loss per epoch: 0.1946, 0.0923, 0.0640, 0.0456, 0.0364, 0.0287, 0.0258, 0.0188, 0.0166, 0.0110.

3.4 Teacher and student on the 100 validation commands

Model	Valid JSON	Intent	Date	Time	Place	Full frame
Teacher	100%	80%	79%	87%	87%	49%
Student	100%	78%	81%	89%	82%	48%

The student comes very close to the teacher. Because it learns from the teacher’s answers, it also learns the teacher’s mistakes.

3.5 Limitations

The assignment asks for CPU training on at most 1,500 records, but I trained on a GPU with 2,765, so this is a deviation. I did not score the untrained Pythia-70M or document a models/v1 version. With only 100 validation commands, one command is one percentage point.

4. Task 3: Containerised Inference Service

The service is written in FastAPI and listens on port 8000. It has three endpoints. POST /parse takes a command and returns the JSON, with the headers X-Model-Version and X-Served-By. GET /healthz returns 200 as long as the program is running. GET /readyz returns 503 until the model has been loaded and 200 after that. Every model output is checked against schema.json before it is returned.

The Docker image holds only the code and pinned libraries, not the model. When the service starts, it downloads the model and tokenizer from SeaweedFS (s3://models/v2/). The settings MODEL_URI, MODEL_VERSION, S3_ENDPOINT and MAX_NEW_TOKENS come from environment variables, and the storage credentials come from a Kubernetes Secret.

4.1 Input and expected output

Input: set an alarm for seven thirty am tomorrow

Expected output: `{"intent":"alarm_set","date":"tomorrow","time":"seven thirty am","place":null}`

4.2 Example request on my laptop

I called the service with a PowerShell request to POST /parse on http://127.0.0.1:8000. The command was “what is the weather in london”. The service answered with the intent weather_query, the place london, and null for date and time, which is what the command contains.

```
(.venv) PS C:\Users\Mukesh Patel\cloud-assignment> $body = @{ text = "what is the weather in london" } | ConvertTo-Json
(.venv) PS C:\Users\Mukesh Patel\cloud-assignment> Invoke-RestMethod -Uri "http://127.0.0.1:8000/parse" -Method Post -ContentType "application/json" -Bo
y $body | ConvertTo-Json -Depth 5

   "intent":  "weather_query",
   "date":  null,
   "time":  null,
   "place":  "london"

(.venv) PS C:\Users\Mukesh Patel\cloud-assignment>
```

Figure 4. PowerShell: request to /parse and the JSON returned by the service.

4.3 Fault injection test

When FAULT_INJECT is set to 1, the service replaces the model output with the text “not json”. The validation caught this, and /parse returned HTTP 500 with an invalid-model-output error. After I removed the variable and rolled the deployment out again, an alarm command returned HTTP 200.

I did not record the image size or the 50-request memory test at 1 CPU and 1 GiB.

5. Task 4: Kubernetes Deployment and Reliability

The cluster is a single-node k3d cluster called cc, running on a Windows laptop with Docker Desktop. SeaweedFS lives in the namespace data and the parser in the namespace app.

Part	Setting
Deployment	Parser Deployment with 2 replicas.
Service and Ingress	A Service in front of the Pods, and an Ingress that sends /parse to it.
CPU and memory	Requests and limits are set on the container in the Deployment file.
Probes	Liveness probe on /healthz and readiness probe on /readyz. A Pod that is still loading the model gets no traffic.
HPA	CPU target 70%, with minimum 2 and maximum 2 replicas.

5.1 Restart and rollback

Restarting the deployment until the Pods were ready took about 32.5 seconds, from one observation. A rollout undo to revision 8 finished successfully, and the service answered requests afterwards.

5.2 Limitations

A single node on one laptop does not show that the system survives the failure of a machine. Since the HPA was fixed at 2 to 2, I did not see it scale from 1 to 2 and back. I also did not measure the rolling update under load, Pod deletion under load, the three cold starts (only one was recorded), or how requests were shared between the replicas.

6. Tasks 5 and 6: Networking, Testing and CI/CD

6.1 Four subnets from 10.20.0.0/16

Subnet	Hosts	Network	Usable range	Broadcast
app	1,000	10.20.0.0/22	10.20.0.1 – 10.20.3.254	10.20.3.255
public	250	10.20.4.0/24	10.20.4.1 – 10.20.4.254	10.20.4.255
data	100	10.20.5.0/25	10.20.5.1 – 10.20.5.126	10.20.5.127
management	20	10.20.5.128/27	10.20.5.129 – 10.20.5.158	10.20.5.159

6.2 Network ACL for the public subnet

Direction	Rule	Action
Inbound	Rule 100: all traffic from 198.51.100.0/24	Deny
Inbound	TCP 80 and 443 from anywhere; TCP 22 only from the management subnet; TCP 1024–65535 from the app subnet (replies)	Allow
Outbound	TCP 8000 to the app subnet; TCP 1024–65535 to anywhere (replies)	Allow

A network ACL is stateless. It does not remember a connection, so the reply to every request needs its own rule. The first matching rule decides, which is why the deny rule for 198.51.100.0/24 has a lower number than the allow rule for port 443. The packet simulator gave the right answer for all 6 of 6 sample packets.

6.3 Testing and CI/CD

Check	Result
Pytest	8 passed
Ruff	All checks passed
Gitleaks	No leaks found
GitHub Actions, run 42360932420	All four jobs passed: lint, unit-tests, evaluate, secret-scan



Figure 5. The GitHub Actions workflow with all jobs passing.

I did not run a deliberate quality-gate failure.

7. Task 7: Performance, Cost and Conclusion

7.1 Performance, resources and cost

Item	Status
Latency (p50, p95), throughput, error rate	Not measured. I do not have a saved Locust result.
CPU and memory of each Pod	Not measured.
Time to become ready after restart	About 32.5 seconds, from one observation.
Cost	Hourly cost = replicas × (CPU limit × 3.50 + memory limit in GiB × 0.45). Cost per 1,000 requests = hourly cost ÷ requests per hour × 1,000. Throughput is needed for this, so I give no figure.
SLO	99% of requests return HTTP 200 during one 2-minute load test. The error budget is 1% of the requests in that run.

7.2 Limitations and challenges

- The cluster has one node, and the HPA was fixed at 2 to 2.
- Training used a GPU and 2,765 records instead of the CPU and 1,500-record rule.
- Full-frame accuracy is 48%, measured on only 100 commands.
- The teacher often did not copy slot values exactly, so 165 records were rejected.
- Several required measurements were not completed.

7.3 Future improvements

- Run Locust, save p50, p95, throughput and memory, then work out the cost and SLO.
- Set the HPA to 1–2 and test scaling, rolling update and Pod deletion under load.
- Retrain on CPU with the first 1,500 records, score the untrained model, and add a failing CI run.

7.4 Conclusion

The project gives a working pipeline from teacher labelling to a two-replica Kubernetes deployment. The student reaches 48% full-frame accuracy against 49% for the teacher, and the tests and CI pass. The main missing parts are the performance and autoscaling measurements.

References

[1] Bastianelli et al., SLURP: A Spoken Language Understanding Resource Package, EMNLP 2020.
[2] Kim and Rush, Sequence-Level Knowledge Distillation, EMNLP 2016.
[3] EleutherAI, Pythia-70M model card.
[4] Microsoft, Phi-3.5-mini-instruct.
[5] Documentation of FastAPI, Kubernetes, k3d and SeaweedFS.
[6] Project repository: https://github.com/mukeshpatel1006/cloud-command-parser-Project